Efficient Data Structures for
RDD-based Apriori on Spark

Luis Alberto Vásquez Vargas
IC – UNICAMP

MO809 – Topics in Distributed Computing
Instructor: Prof. Luiz Fernando Bittencourt

l322989@dac.unicamp.br
April 7, 2026

0. Outline

1. The Base Study
2. Introduction & Motivation
3. The FIM Problem
4. Infrastructure Evolution
5. Literature Overview
6. Proposed Data Structures
7. Experimental Results
8. Conclusion
9. References

SECTION 1

The Base Study

1. The Base Study

Original Research Paper

A Data Structure Perspective to the RDD-based Apriori Algorithm on Spark

- **Source:** *International Journal of Information Technology* (Springer), 2019.
- **Keywords:** Parallel Algorithms, Big Data, Apriori, Spark, RDD.
- **Authors & Affiliations:**
  - **Pankaj Singh & P. K. Mishra:** Dept. of Computer Science, BHU.
  - **Sudhakar Singh:** Dept. of Electronics & Comm., Univ. of Allahabad.
  - **Rakhi Garg:** Mahila Maha Vidyalaya, BHU, Varanasi.

Pankaj Singh

Sudhakar Singh

P. K. Mishra

Rakhi Garg

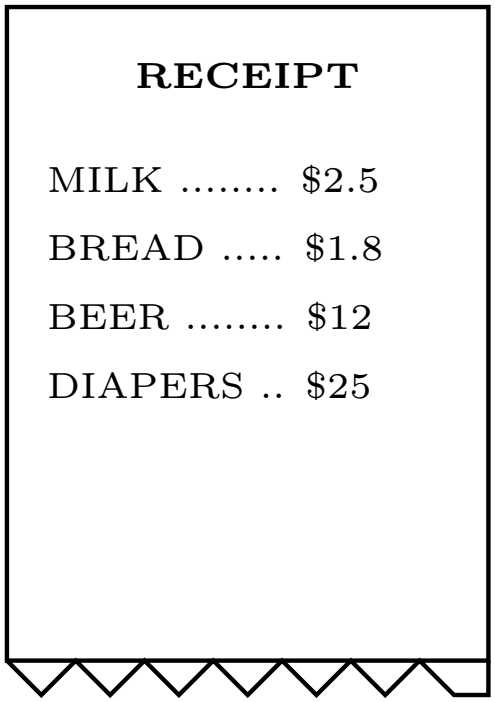
Focus: Comparative efficiency of **Hash Tree**, **Trie**, and **Hash Table Trie** in distributed Spark environments.

SECTION 2

Introduction & Motivation

2. The Market Basket Problem: From Paper to Patterns

1. Physical Transactions



Digitize

2. Digital "Baskets"

T1: {Milk, Bread, Beer, Diapers}

T2: {Milk, Eggs, Bread}

T3: {Beer, Diapers, Chips}

⋮

3. Discover Frequent Itemsets

Search Space: 2^n Subsets

2. Mining Actionable Patterns

Examples of Association Rules:

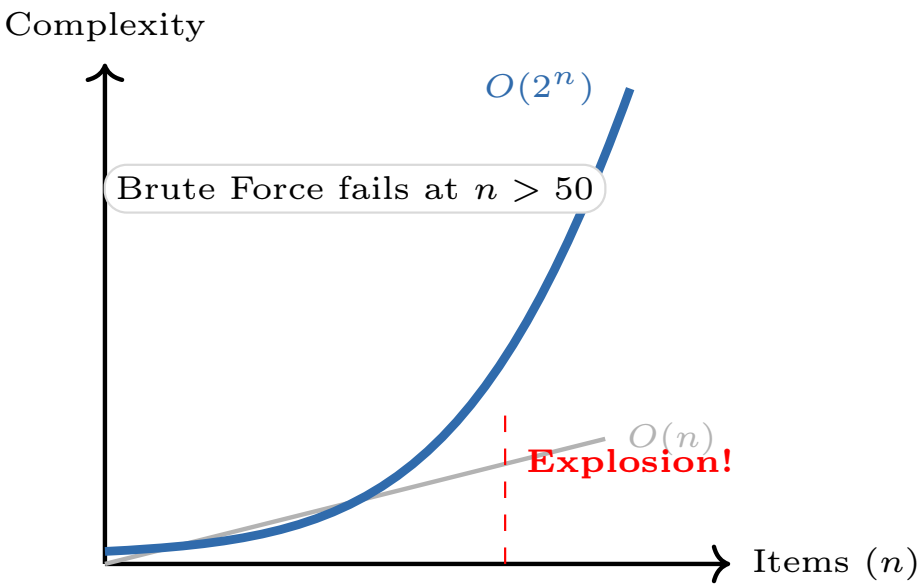
- $\{Milk, Bread\} \implies \{Eggs\}$
- $\{Beer\} \implies \{Diapers\}$
- $\{Wine\} \implies \{Cheese\}$
- $\{Laptop\} \implies \{Mouse, Keyboard\}$

The Data Mining Goal

Find all subsets of items (itemsets) that appear in at least s transactions within a massive dataset.

- **Applications:** Cross-selling, inventory management, and store layout optimization.

2. The Technical Challenge: Complexity



$$n = 100 \implies 2^{100} \text{ subsets.}$$
$$\approx 3,000\times \text{ Age of Universe by Brute Force.}$$

Power Set for $n = 4$

Size	Subsets
$k = 0$	$\emptyset$
$k = 1$	$\{a\}, \{b\}, \{c\}, \{d\}$
$k = 2$	$\{ab\}, \{ac\}, \{ad\}, \ldots$
$k = 3$	$\{abc\}, \{abd\}, \ldots$
$k = 4$	$\{abcd\}$

Total: 16 Combinations

Adding 1 item doubles the work!

The Bottleneck: Exponential $O(2^n)$ combinatorial complexity makes massive-scale association rule mining compute-bound.

2. The Infrastructure Solution

Apache Spark RDDs

- **Hardware Evolution:**
  - We use **Apache Spark RDDs**: the distributed "Ferrari" of in-memory computing.
- **The Bottleneck:**
  - Legacy structures (Hash Trees) act like a **traffic jam** in a high-speed engine.

The Core Thesis

Infrastructure alone cannot solve $O(2^n)$. To exploit Spark's speed, we must replace "Disk-era" structures with memory-optimized **Tries**.

SECTION 3

The FIM Problem

3. Origins and Definition

- **Market Basket Analysis:** The historical root of FIM.
- **The Item Base:** Let $\mathcal{B} = \{i_1, \ldots, i_n\}$ be the set of all possible items.
- **Itemsets:** Any subset $I \subseteq \mathcal{B}$.
- **Transactions:** A database $\mathcal{T} = (t_1, \ldots, t_m)$ where each transaction t_k is an itemset.

Frequent Itemset Mining (FIM)

The task of discovering all itemsets I that appear in the database $\mathcal{T}$ with a frequency (support) greater than or equal to a threshold s_{min}:

$$s_{\mathcal{T}}(I) = |\{k \in \{1, \ldots, m\} \mid I \subseteq t_k\}| \geq s_{min}$$

3. Illustrative Example

Transaction Database ($\mathcal{T}$)

0: $\{a, d, e\}$ 5: $\{a, c, d\}$

1: $\{b, c, d\}$ 6: $\{b, c\}$

2: $\{a, c, e\}$ 7: $\{a, c, d, e\}$

3: $\{a, c, d, e\}$ 8: $\{b, c, e\}$

4: $\{a, e\}$ 9: $\{a, d, e\}$

Frequent Itemsets ($s_{min} = 3$)

0 items	**1 item**	**2 items**	**3 items**
$\emptyset : 10$	$\{a\} : 7$	$\{a, c\} : 4$	$\{a, c, d\} : 3$
	$\{b\} : 3$	$\{a, d\} : 5$	$\{a, c, e\} : 3$
	$\{c\} : 7$	$\{a, e\} : 6$	$\{a, d, e\} : 4$
	$\{d\} : 6$	$\{b, c\} : 3$	
	$\{e\} : 7$	$\{c, d\} : 4$	
		$\{c, e\} : 4$	
		$\{d, e\} : 4$	

3. The Antimonotonicity Rule

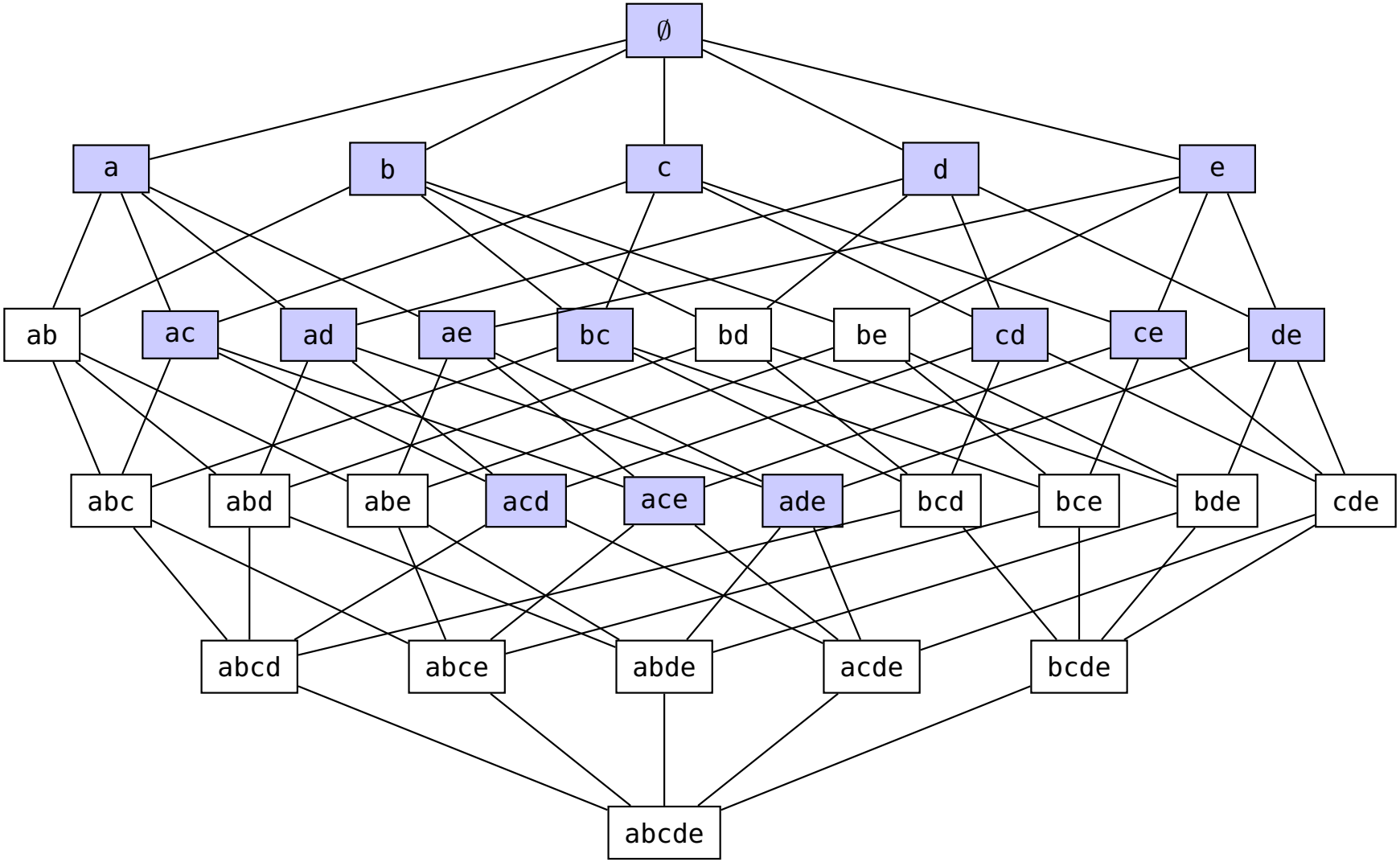
- **The Core Principle:** The support of an itemset can never exceed the support of its subsets.
- **Formal Rule:** $\forall I, J : I \subseteq J \implies s_{\mathcal{T}}(I) \geq s_{\mathcal{T}}(J)$.

The Pruning (Downward Closure) Property

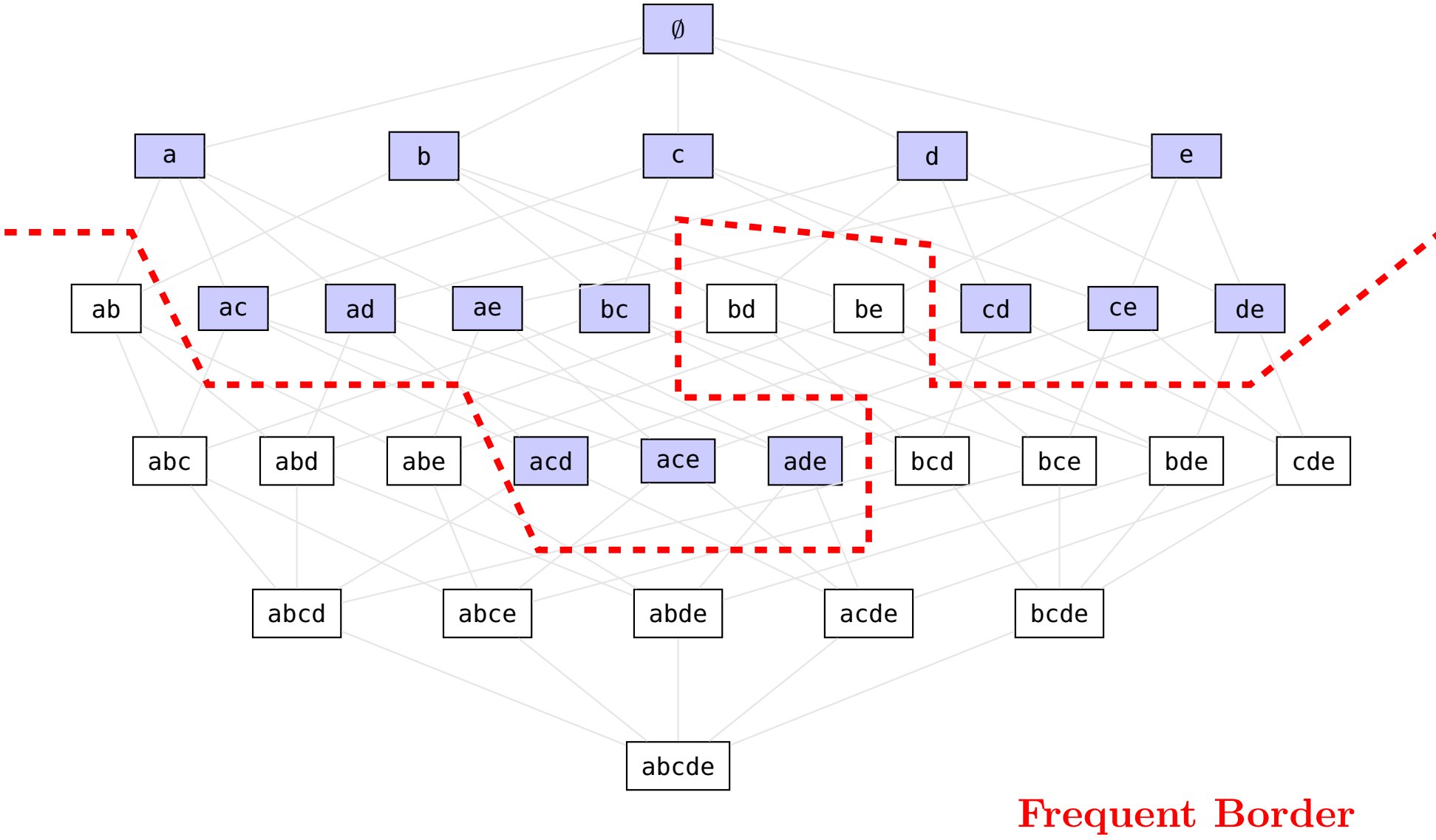
If an itemset is **infrequent**, all of its supersets are also **infrequent**.

- **Search Strategy:** We "cut" the search lattice. If a node fails s_{min}, its entire downward branch is discarded.
- This defines a spatial boundary in the lattice: the **Frequent Border**.

3. The Complete Hasse Diagram



3. The Frequent Border (Spatial Boundary)



3. The Apriori Algorithm

Apriori is a **level-wise** search algorithm. It uses a "generate-and-test" strategy:

1. **K=1:** Find all frequent items (e.g., $\{a\}, \{b\}$).
2. **Join Step:** Generate candidates of size $k+1$ from frequent k-itemsets.
3. **Prune Step:** Use the **Antimonotonicity Rule** to delete invalid candidates.
4. **Count Step:** Scan the database to calculate actual support.

Repeat until no more candidates can be generated.

3. The Join Step: $L_k \to C_{k+1}$

The Prefix-Join Rule

Join frequent itemsets $p, q \in L_k$ **only if** they share the same prefix of length $k-1$:

$$p = \{i_1, \ldots, i_{k-1}, i_k\} \quad q = \{i_1, \ldots, i_{k-1}, i_{k'}\}$$

$$\text{Candidate } c = \{i_1, \ldots, i_{k-1}, i_k, i_{k'}\}$$

Why this works:

- It generates the *minimum* necessary candidates.
- Example: If $\{a, c\}$ and $\{a, d\}$ are frequent, $\{a, c, d\}$ is a potential candidate.
- If $\{a, c\}$ is frequent but $\{c, d\}$ is **not**, the join logic naturally skips $\{a, c, d\}$.

3. Why is this slow in practice?

The Problem: Searching a Giant Haystack

- Even with pruning, we might have **millions** of candidate itemsets.
- Every transaction requires a frequency check: *"Is this candidate in here?"*

The "Librarian" Analogy

Imagine looking for a specific book in a massive, poorly indexed library:

- **The Task:** Finding 5 books out of 1,000,000 possibilities.
- **The Waste:** Walking down 50 hallways just to find an empty shelf.
- **The Reality:** We spend **90%** of the time "walking" (memory access) and only **10%** "reading" (actual counting).

Goal: We need a way to "teleport" directly to the right shelf.

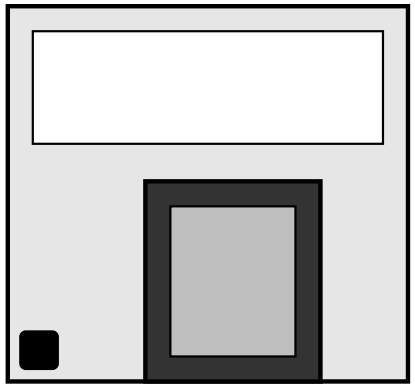
SECTION 4

Infrastructure Evolution

4. The Context: Why now and not before?

Before 2004, "Big Data" was handled by massive, expensive vertical mainframes.

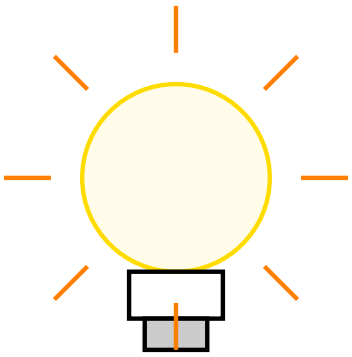
- **The "Single Giant" (Vertical):** Hitting the physical and financial ceiling of a single machine.
- **The "Army of Ants" (Horizontal):** Linking 1,000 "commodity" PCs to act as one.



Economic Necessity

In the 90s, RAM was >**$50/MB**. We switched because scaling vertically became **bankrupting**.

4. Google's Contributions (2004)



The *"MapReduce"* paper by Dean & Ghemawat:

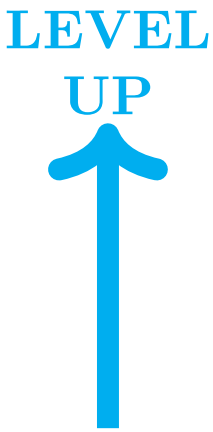
- **Abstraction:** Simplified distributed math into Map/Reduce.
- **Fault Tolerance:** Hardware failure is now a software problem.
- **Data Locality:** "Move the computation, not the data."

The Catalyst for Innovation

By sharing this blueprint, Google allowed the community to build **Apache Hadoop**, moving beyond the "Single Giant" era.

4. From the "Basement" to the "Desk"

- **Hadoop Era (2006):** Relied on Disk. Like a librarian walking to the **basement** for every book.
- **Spark Era (2014):** Uses RAM (RDDs). The librarian keeps everything on the **desk**.



Feature	**Hadoop (Disk)**	**Spark (RAM)**
Processing Speed	1x	**100x Faster**
Data Access	Sequential	**Instant (RDD)**

4. Spark Architecture & The New Bottleneck

- **The Setup:** Database partitioned across the cluster's RAM.
- **The Pattern:** Candidates (C_k) are broadcast to every worker.

The CPU Wall

Data is now in RAM, so the disk is no longer the problem. The new bottleneck is the **CPU** searching millions of candidates per second.

*Coming up: A deep dive into the **Data Structures** that will solve this bottleneck...*

SECTION 5

Literature Overview

5. Literature Overview: FIM Evolution

- **The Classic Era (Sequential):**
  - **Apriori (1994):** The foundation. Used Hash Trees to store candidates.
  - **FP-Growth (2000):** Eliminated candidate generation using a prefix-tree (FP-Tree), but is notoriously difficult to parallelize efficiently.
- **The Distributed Era (Disk-based):**
  - **PFP (2008):** Parallel FP-Growth on MapReduce. Heavy Disk I/O overhead due to the nature of Hadoop.
- **The Modern Era (In-Memory):**
  - **Spark-based Apriori:** Focuses on RDD persistence to avoid re-reading data.
  - **The Gap:** Most Spark implementations still rely on standard Hash Trees or simple Arrays, failing to optimize the CPU-to-Memory bottleneck.

5. Literature Overview: Data Structure Benchmarks

- **Hash Tree (HT):**
  - Standard for decades.
  - *Issue:* High branch misprediction and poor cache locality in modern CPUs.

- **Trie (Prefix-Tree):**
  - Fast traversal, no hashing overhead at each node.
  - *Issue:* Can become very deep and sparse, consuming significant memory.

- **The Research Question:**

  *Can a hybrid approach—the **Hash Table Trie**—outperform both in a distributed RDD environment where memory is abundant but CPU cycles are precious?*

SECTION 6

Proposed Data Structures

6. Data Structures: The Hash Tree

- **Structure:** Internal nodes contain hash tables; leaves store candidate itemsets.
- **Traversal Logic:**
  1. Hash the i-th item of a transaction.
  2. Follow the corresponding branch.
  3. Repeat until a leaf is reached and verify the candidate.

- **The Spark Bottleneck:**
  - **High Collision:** Multiple itemsets mapping to the same leaf force slow linear searches.
  - **CPU Overhead:** Repeated hashing at every level is expensive when scaled across RDD partitions.

6. Data Structures: The Trie (Prefix Tree)

- **Structure:** A rooted tree where each node represents a single item.
- **Mechanism:**
  - Paths from root to leaf represent full itemsets.
  - Common prefixes are shared, reducing redundant storage.

- **Advantage in RDDs:**
  - **Zero Hashing:** Navigates purely by pointers/references.
  - **Predictable Traversal:** If a prefix isn't in the transaction, the entire branch is pruned instantly.
  - *Trade-off:* Searching for a child node among many siblings can become $O(k)$ where k is the alphabet size.

6. Data Structures: The Hash Table Trie

- **The Hybrid Concept:**
  - A Trie where each node uses a **local Hash Table** to store its children.
- **Why it dominates in Spark:**
  - **Constant Time ($O(1)$):** Finding the next item in a sequence doesn't require scanning a list of siblings.
  - **CPU Efficiency:** It combines the pruning power of a Trie with the lookup speed of Hashing.
  - **Memory-Speed Tradeoff:** Uses more RAM than a standard Trie, but in an RDD environment, RAM is exactly what we have to spare.

The Verdict

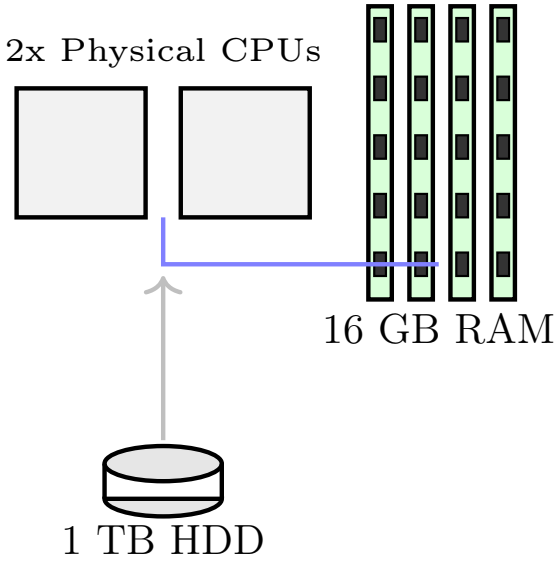
The Hash Table Trie fixes the "branching bottleneck" of standard Tries while maintaining better cache locality than traditional Hash Trees.

SECTION 7

Experimental Results

7. Experimental Results: Hardware Specifications

- **PC Build (Workstation):**
  - **CPU:** Intel Xenon CPU E5-2620@2.10 GHz with 24 cores *(Dual-Processor setup)*.
  - **RAM:** 16 GB DDR3 *(Inferred Quad-Channel configuration)*.
  - **Storage:** 1 TB of disk storage *(Inferred HDD for HDFS layer)*.

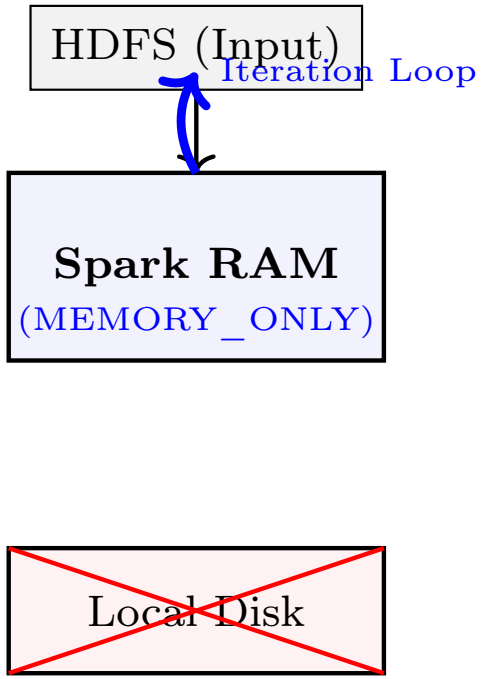


Hardware Capacity

The dual-processor setup provides 24 execution threads, while the quad-channel memory configuration ensures high data transfer speeds between the RAM and the CPUs.

7. Experimental Results: Software & Memory Mode

- **Software Stack:**
  - **OS:** Ubuntu 14.04 (64-bit).
  - **Frameworks:** Spark 2.1.1, Hadoop 2.6.0.
  - **Language:** Java 7 / Scala 2.11.8.
- **Memory Mode:**
  - **Level:** `MEMORY_ONLY`.
  - **Strategy:** RDDs are cached in RAM.
  - **Impact:** Bypasses Disk I/O bottlenecks during L_k generation.



7. Experimental Results: Dataset Properties

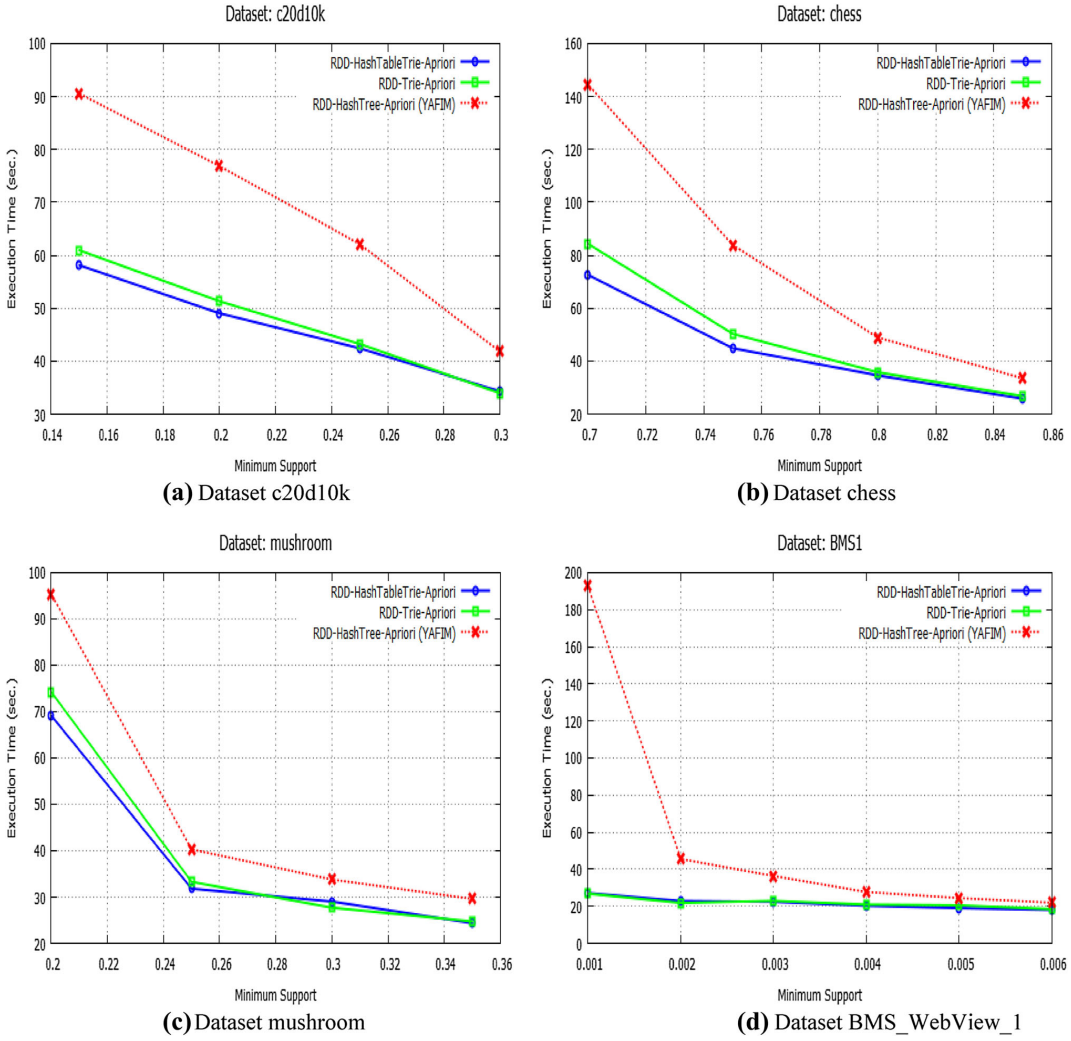
Table 1: Datasets used in the experiments

Dataset	Transactions	Items	Avg. Width	Characteristic
c20d10k	10,000	192	20	Dense
chess	3,196	75	37	Extremely Dense
mushroom	8,124	119	23	Dense
BMS_WebView_1	59,602	497	2.5	Sparse
BMS_WebView_2	77,512	3,340	5	Sparse
T10I4D100K	100,000	870	10	Sparse
T40I10D100K	100,000	1,000	40	Dense

Dataset Diversity

The benchmark selection spans from sparse transaction logs to dense datasets, allowing for a robust evaluation of the HTT architecture across different data characteristics.

7. Performance Analysis: Datasets (a) to (d)

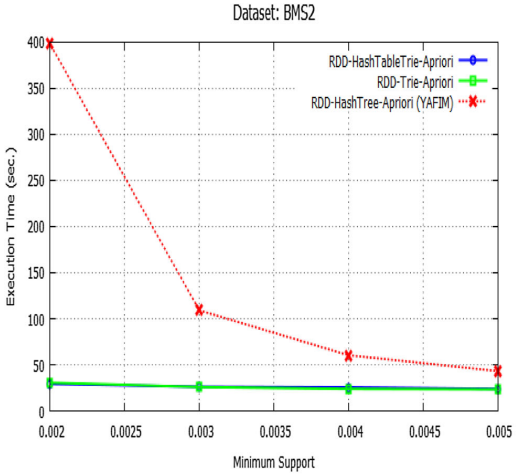


(a) Dataset c20d10k **(b)** Dataset chess

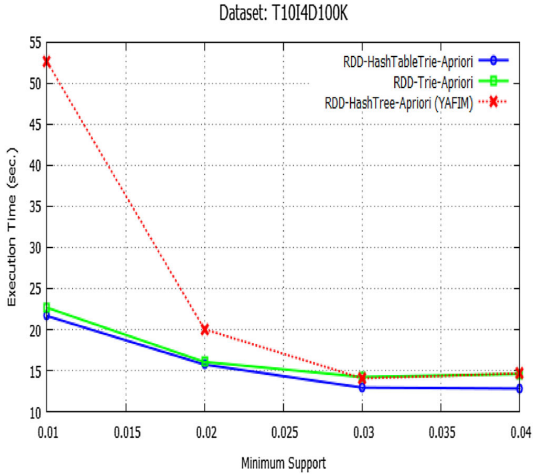
(c) Dataset mushroom **(d)** Dataset BMS_WebView_1

- **Observation:** On dense datasets like *Mushroom* and *Chess*, HTT maintains a steady **2x speedup** by optimizing CPU cache usage during RDD lookups.

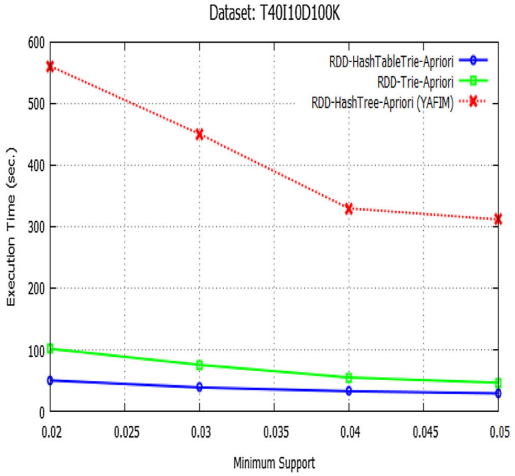
7. Performance Analysis: Datasets (e) to (g)



(e) Dataset BMS_WebView_2



(f) Dataset T10I4D100K



(g) Dataset T40I10D100K

- **Observation:** For sparse datasets (BMS/T10), the Trie-based architecture achieves a significant **8x performance gain**.

7. Experimental Results: Summary Table

Relative Speedup vs. Hash Tree (HT) Baseline

Dataset	HT	Trie	HTT	Characteristic
c20d10k	1.0x	1.7x	**1.8x**	Dense
chess	1.0x	1.9x	**2.1x**	E. Dense
mushroom	1.0x	1.9x	**2.1x**	Dense
BMS_WebView_1	1.0x	8.1x	**8.4x**	Sparse
BMS_WebView_2	1.0x	8.1x	**8.4x**	Sparse
T10I4D100K	1.0x	5.9x	**6.2x**	Sparse
T40I10D100K	1.0x	5.8x	**6.1x**	Dense

The RDD Conclusion

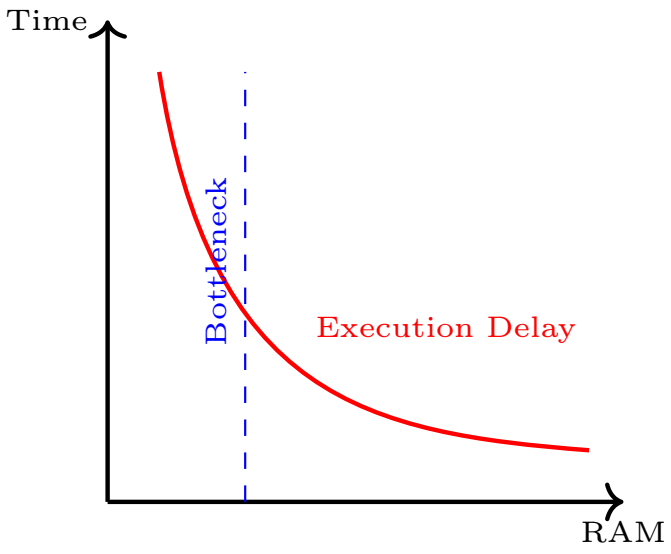
The results validate that within Spark, the computational efficiency of $O(1)$ lookups in the Hash Table Trie (HTT) justifies its memory footprint.

SECTION 8

Conclusion

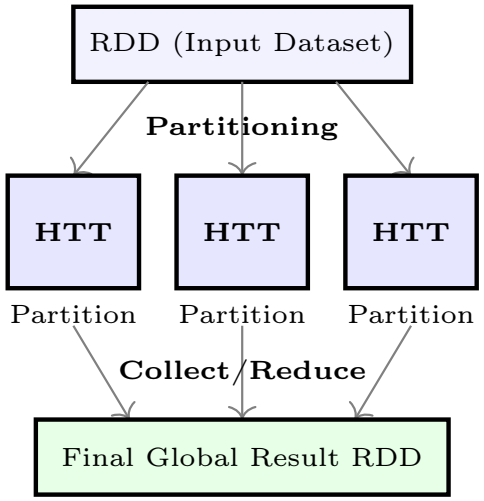
8. Conclusion: The Sequential Context

- **Theoretical Expectation:**
  - The **Hash Table Trie (HTT)** was designed to accelerate itemset traversal by combining Trie logic with hashing.
- **Previous Practical Failure:**
  - In earlier sequential implementations, where **memory was a strict bottleneck**, the additional **cost of hashing** caused it to underperform.
  - It was often documented as the least efficient variant in non-distributed settings.

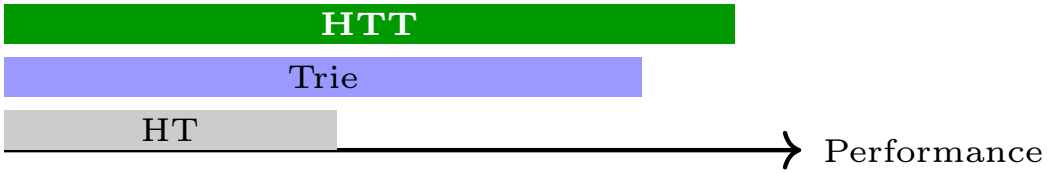


8. Conclusion: The Spark Paradigm Shift

- **Distributed Advantage:**
  - Within Spark's **distributed memory (RDD)** architecture, the **Hash Table Trie (HTT)** finally realizes its theoretical efficiency.



- **Benchmark Results:**
  - Experimental data shows it performs **significantly better** than the traditional **Hash Tree (HT)**.
  - It maintains a slight edge over the standard **Trie** across most datasets.



8. Conclusion: Final Verdict

- **In-Memory Optimization:**
  - Memory-optimized structures are the key to high-performance FIM on Spark.

- **Summary:**
  - The **Hash Table Trie (HTT)**, once considered the "worst" in sequential settings, is now the most robust choice for RDD-based Apriori.
  - Infrastructure alone (Spark) isn't enough; the underlying **data structure** must evolve to match the hardware.

Key Takeaway

Sequential "Worst" $\rightarrow$ Distributed "Best"

SECTION 9

References

9. References I

P. Singh, S. Singh, P. K. Mishra, and R. Garg.
A data structure perspective to the RDD-based Apriori algorithm on Spark.
Int. J. Inf. Technol., 14(3):1585–1594, 2019.

M. J. Zaki.
Scalable algorithms for association mining.
IEEE Trans. Knowl. Data Eng., 12(3):372–390, 2000.

C. Borgelt.
Frequent item set mining.
WIREs Data Mining Knowl Discov, 2:437–456, 2012.

B. Goethals.
Survey on Frequent Pattern Mining.
Helsinki Institute for Information Technology (HIIT), 2003.

9. References II

C. Borgelt.
Efficient Implementations of Apriori and Eclat.
Workshop on Frequent Itemset Mining Implementations (FIMI), 2003.

R. Agrawal and R. Srikant.
Fast algorithms for mining association rules.
Proc. 20th Int. Conf. VLDB, 487–499, 1994.

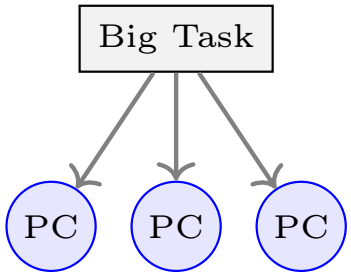
H. B. Abdalla, Y. Kumar, Y. Zhao, and D. Tosi.
A Comprehensive Survey of MapReduce Models for Processing Big Data.
Algorithms, 13(11), 2020.

C. C. Aggarwal.
Data Mining: The Textbook.
Springer, 2015.

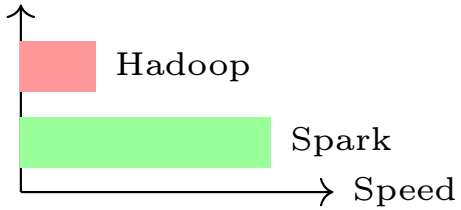
Questions?

Obrigado! ☺

Distributed Work



Speed Comparison



Luis Alberto Vásquez Vargas
IC – UNICAMP

l322989@dac.unicamp.br